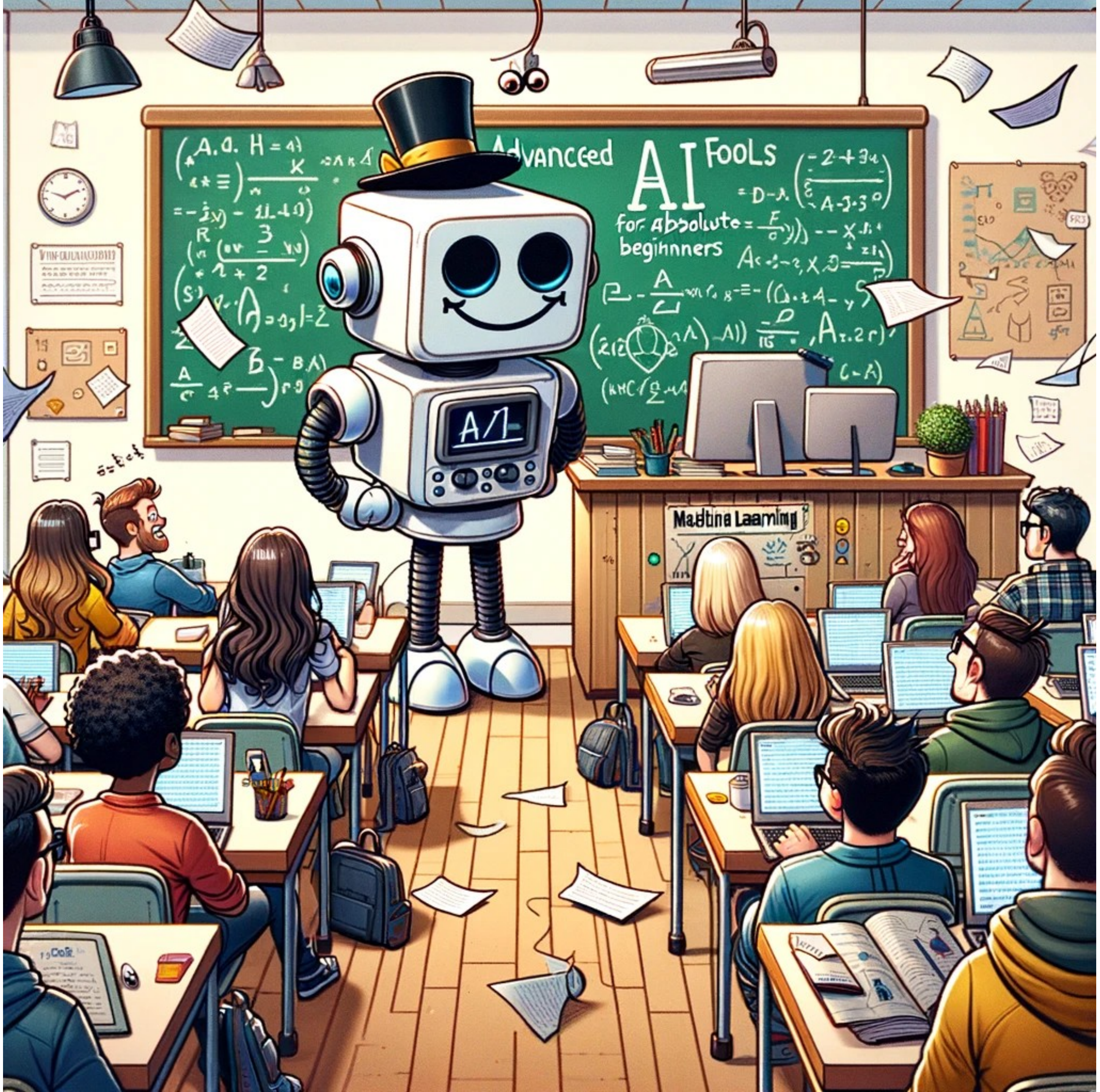


CS 229: Machine Learning

Spring quarter – March 30, 2026

Instructor: Greg {Ver Steeg

TA:

Partha Thakuria

Karpathy:
“LLM research is [about] summoning ghosts”

Amodei (Anthropic CEO):
“Imagine a country of geniuses in a datacenter”

Just next-token prediction!

And we are *just* a bundle of neurons

What makes it work?

- Powerful enough model
- Trained with large and complex enough data
- Optimizing right thing in right way, and
- Engineered to scale

First week or two – getting into some details

Some other topics:

NLP

- Tuning (LoRA) vs in context learning
- Scaling laws

Vision self-supervision/contrastive

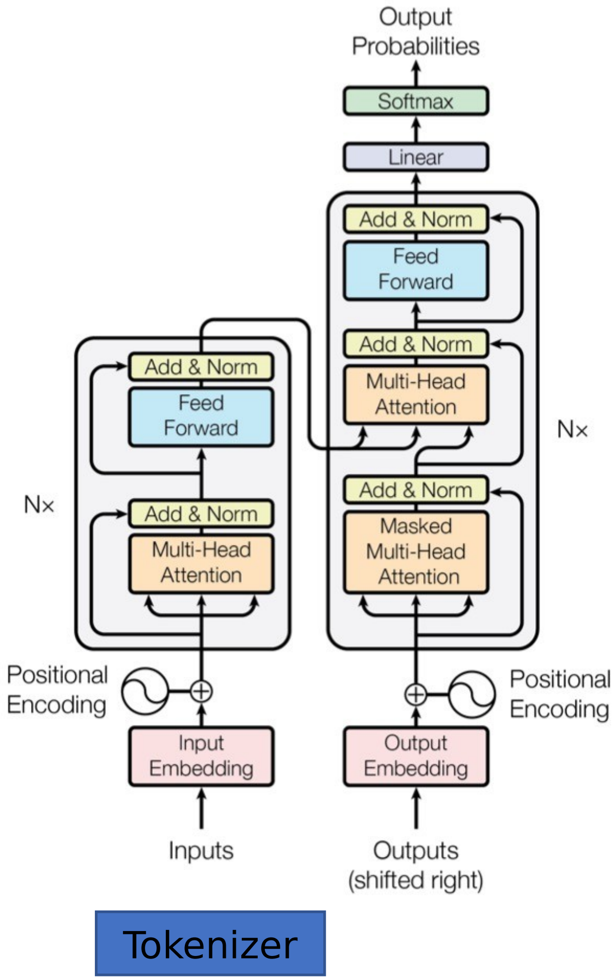
Uncertainty quantification

Deep Bayes

Optimization dynamics

Gen. models (focus on flow matching)

Graph neural nets



Course logistics

On course website

https://elearn.ucr.edu/courses/228278

Syllabus is there

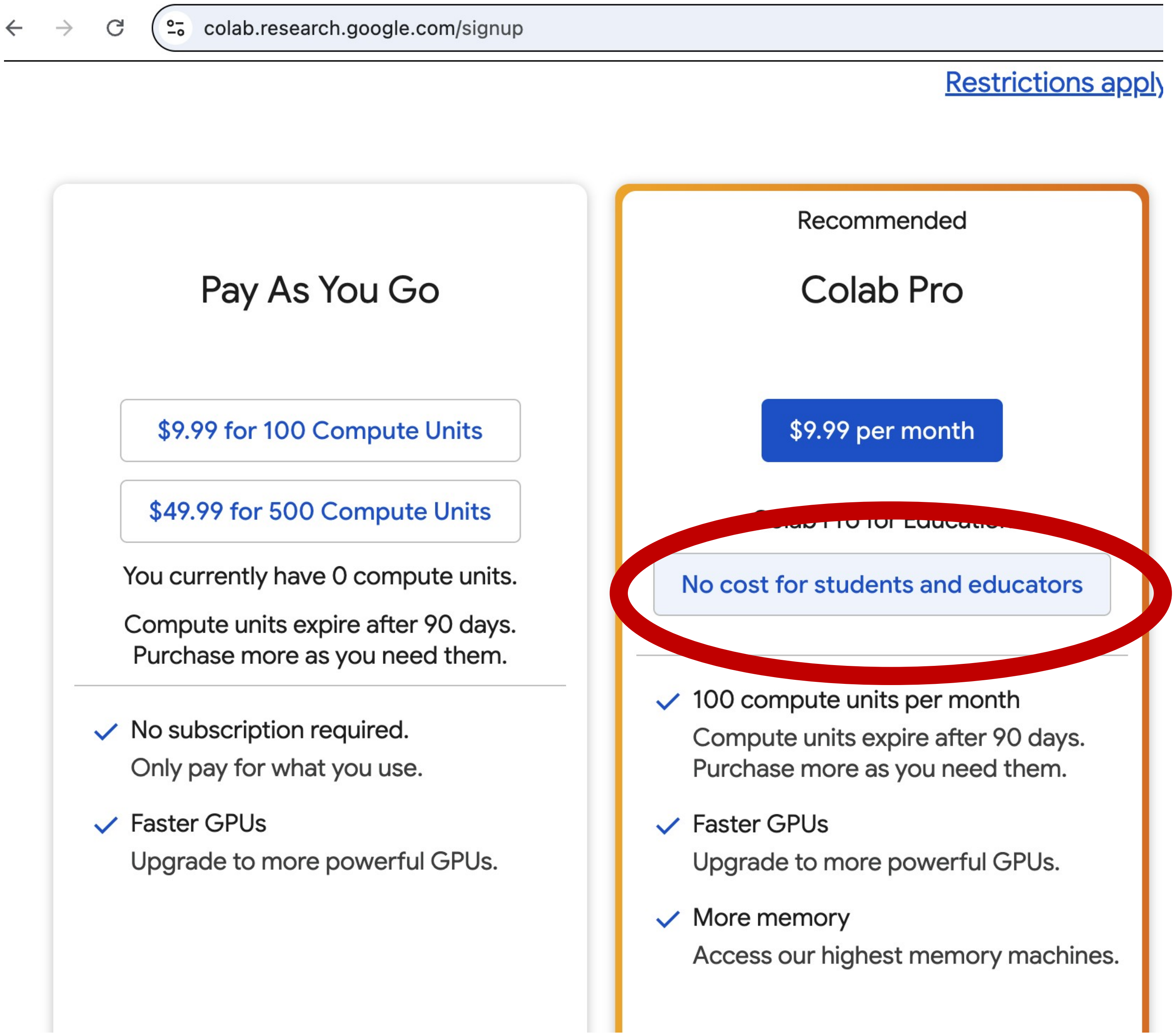
Announcement via Canvas email list

(*You should have gotten one already! If not email me to be added.)*

3.5 Homeworks

Homework 0 – 10% (*Discuss shortly*)
Homework 1 – 15% (CalcGPT)
Homework 2 – 15% (Uncertainty quantification)
Homework 3 – 15% (Flow matching w/ ViT)
55%

HW 1-3 will have some significant but challenging Extra Credits for people who want to go deep.



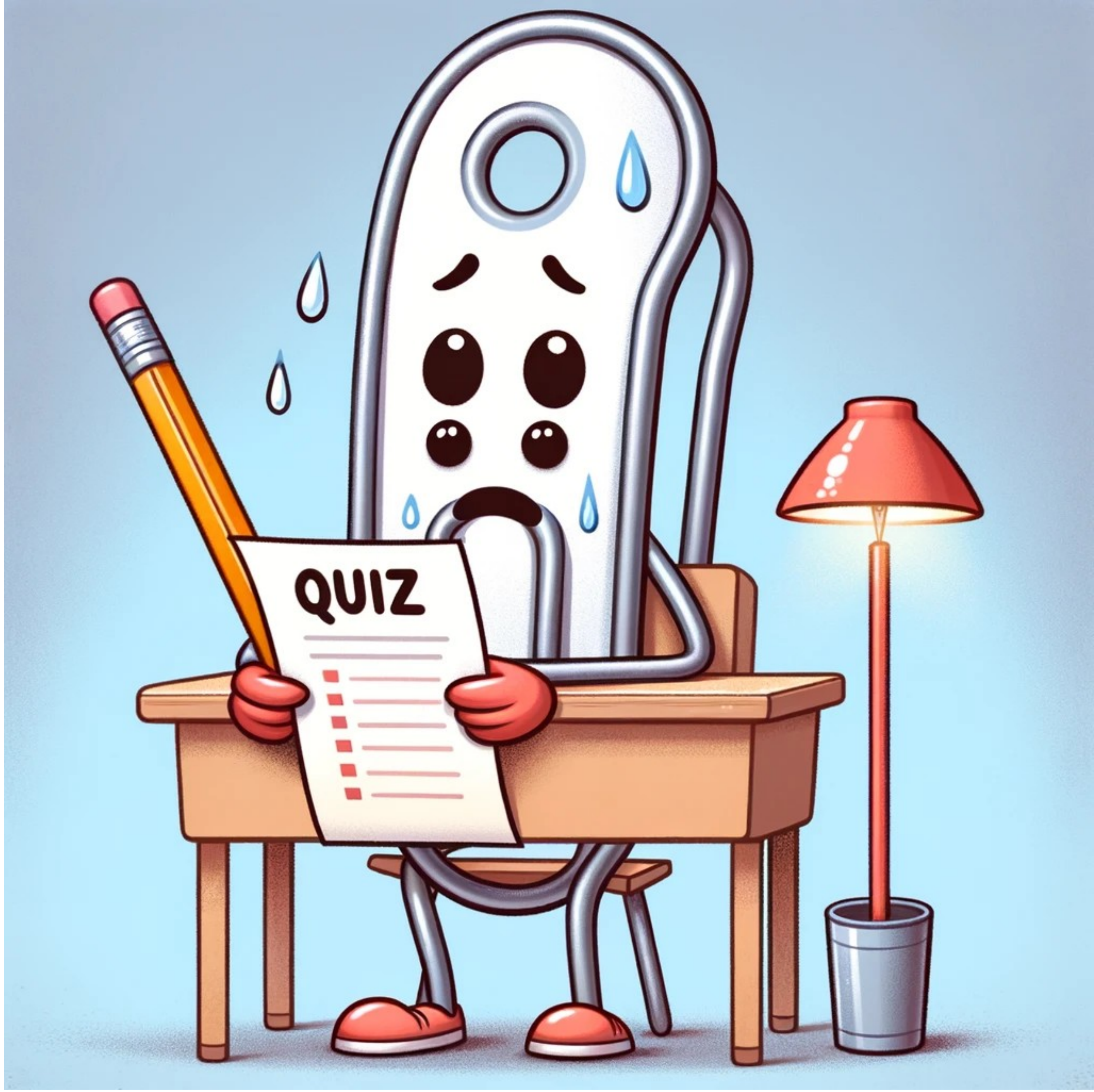
Free Colab Pro for students! It has serious GPUs!

Quizzes – don't study (20%)

- Frequent low stakes feedback for you and me
- Do it without help – value of being wrong

- Practice for final

- New ideas
- *** Challenge problems



Final (20%)

Variations of quiz questions

GPT-4 Prompt: I need a final version of the anthropomorphized paper clip. This time he's taking a final exam and his stress levels are through the roof - ten times the anxiety of the previous pictures.

Generative AI policy / thoughts

- **Yes:** Explain
- **No:** Auto-generate

Experimental code comprehension test (5%)

Background

Quiz 0: Due Saturday. I’ll overview results on Monday. Meant to gauge background knowledge

HW 0: due April 10 (Friday, in 12 days!)

If you feel like you are not prepared for the class, consider auditing and letting someone on the waitlist join.

Canvas-Modules

Video coding demos

Below are some links to the coding demos I did in CS 224. The actual code is in the "demos" dropbox folder also linked in modules.

You may have to skip the first part of the lecture, where I usually talked about quiz results, to get to the coding part.

Note: they are in *reverse* order, so the bottom video is week 1.

Week 1 - start at 61:35 - tensor basics in pytorch

Week 2 - start at 34:50 - models in pytorch, tokenizer

Week 3 - start at 31:20 - Cross validation (skippable)

week 4 - start at 9:31 - Multilayer perceptron, Loss, SGD (Start here if you have some experience)

week 5 - start at 17:55 - Debugging neural nets, MLP training demo (Start here if you have lots of experience and want a refresher)

Slides

Posted from 1-100 minutes before class

Link is on canvas

Books and papers

Books with good background on various topics online – we won’t closely follow one

I’ll email some papers to “read” that I’ll cover in class

Nano banana

Office hours – see syllabus

Office hours:

1:50pm-3:00pm Monday (right after class - we can walk to my office MRB 4114 for longer chats)

6:00-7:00 pm Thursday on Zoom

TA Office hours:

10:00-11:00 am Tuesday, WCH 110

6:00-7:00 pm Friday on Zoom

eriksen,anara,lucie

jieron,slan,keyanvexy

HW 0: AI generated name detector

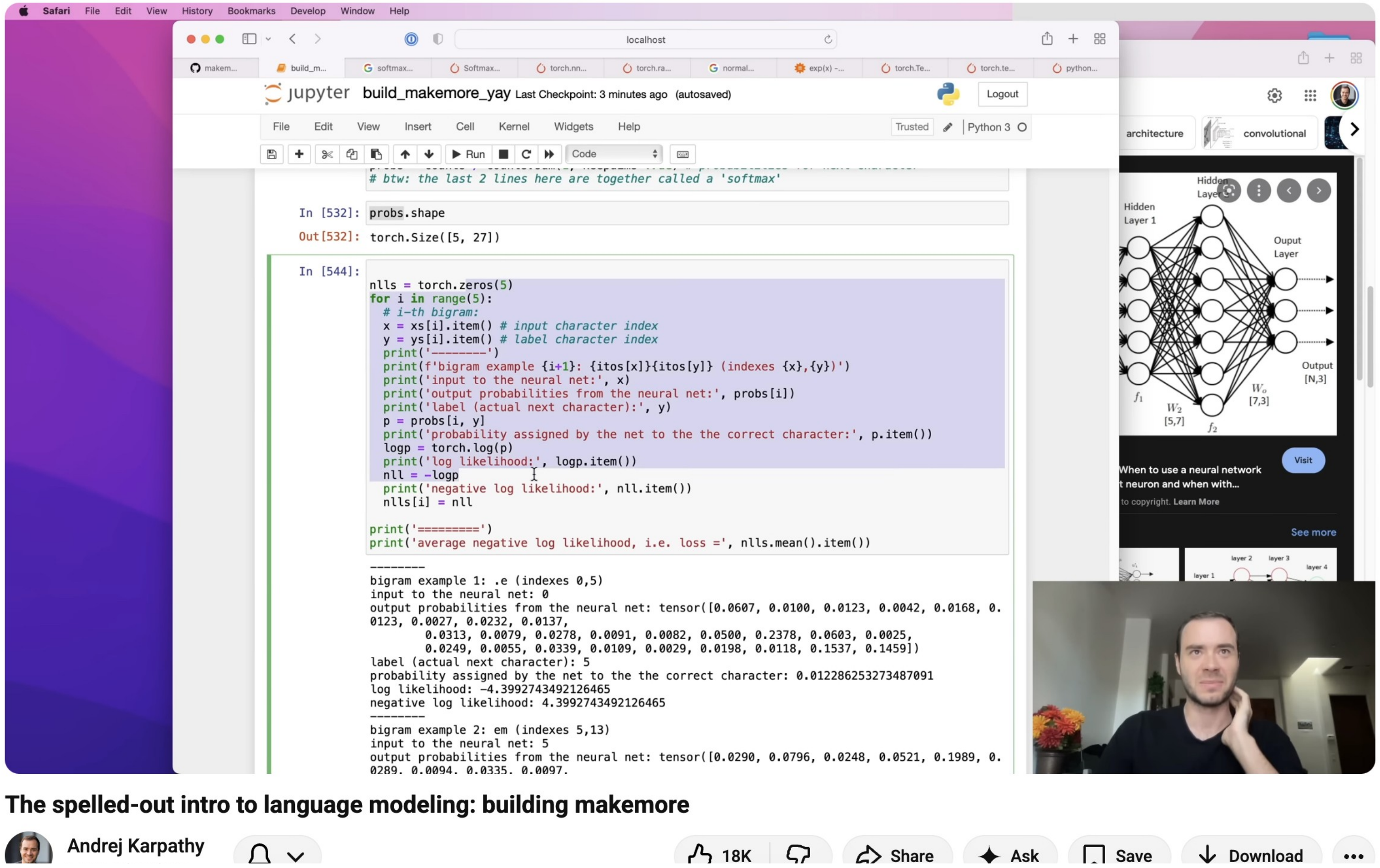
- Catch up on basic Pytorch if you are new to it
- Dive into some LLM details (tokenization, padding, batch processing) for the rest of us

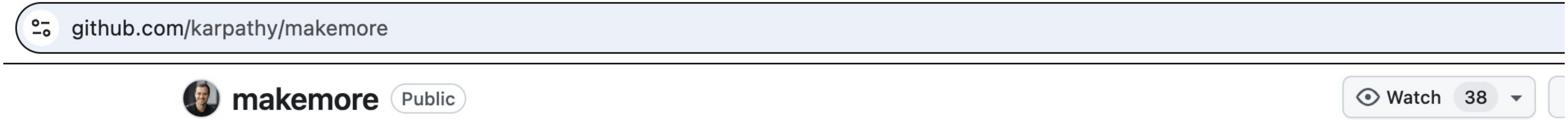
Data comes from Karpathy’s “makemore” project

```
Social security database:
eriksen,anara,lucie…

Generated:
jieron,slan,keyanvexy…
```

Also check out his
“nanoGPT”
“nanoChat”
“Autoresearch”!

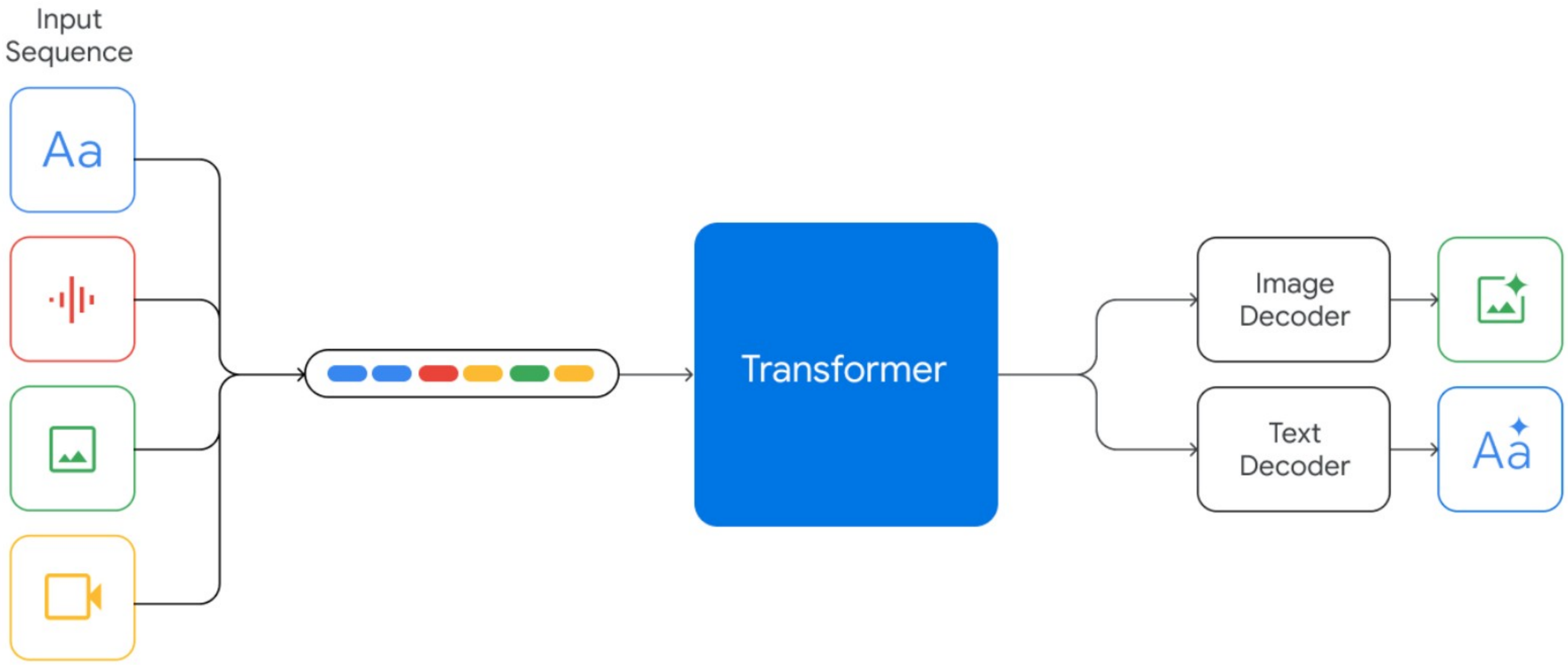




How to process these? `jieron, slan, keyanvexy …`

General problem – turn everything into numbers, turn numbers back into everything

Multimodal LLMs



Turn text into a sequence of numbers

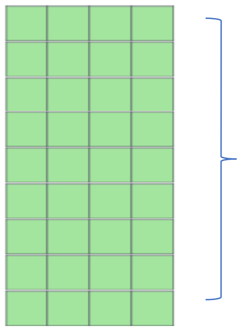
Keyanvexy
11, 5, 25, 1, 14, 22, 5, 24, 25

Then embed as *float vectors*

E_{11}, E_5, E_{25}, …

Size of the final representation?

(length of sequence, d)



Multiple sequences in a tensor?

keyanvexy

11, 5, 25, 1, 14, 22, 5, 24, 25

greg

7, 18, 5, 7

greg

7, 18, 5, 7, 0, 0, 0, 0, 0 **Pad to same length. Pad_token_id=0**

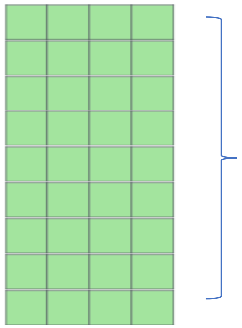
```
input_ids = torch.tensor([11,  5, 25, 1, 14, 22, 5, 24, 25],
                        [ 7, 18,  5, 7,  0,  0, 0,  0,  0]])
```

Then embed

Size of the final representation?

(Batch size, max length of sequence,

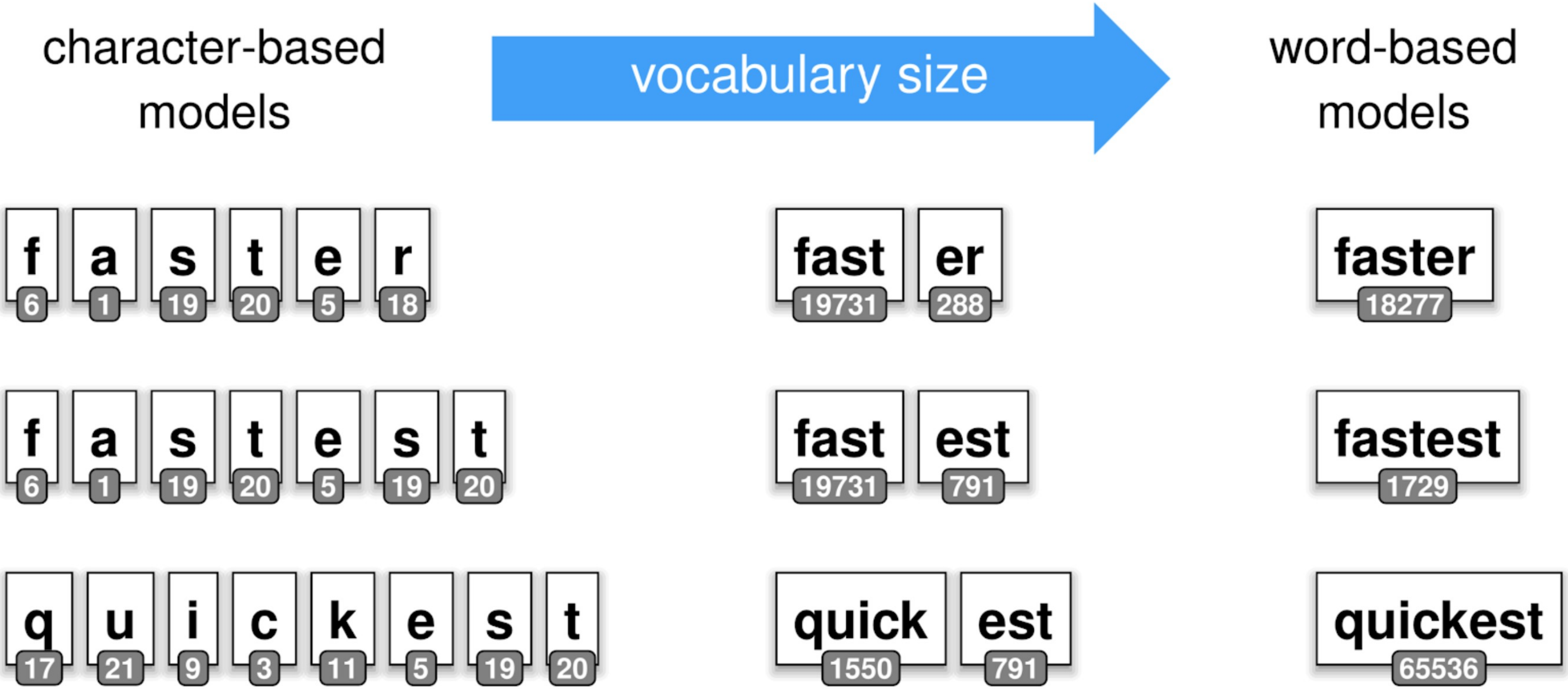
d – embed dimension



Row for each token in vocabulary

E, Embedding table

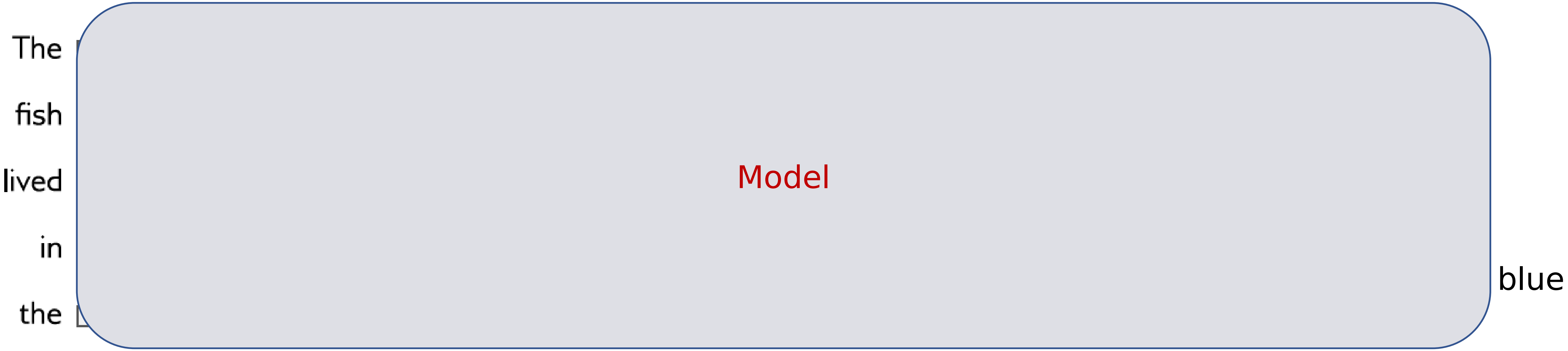
Best way to tokenize? Byte-Pair Encoding (BPE)



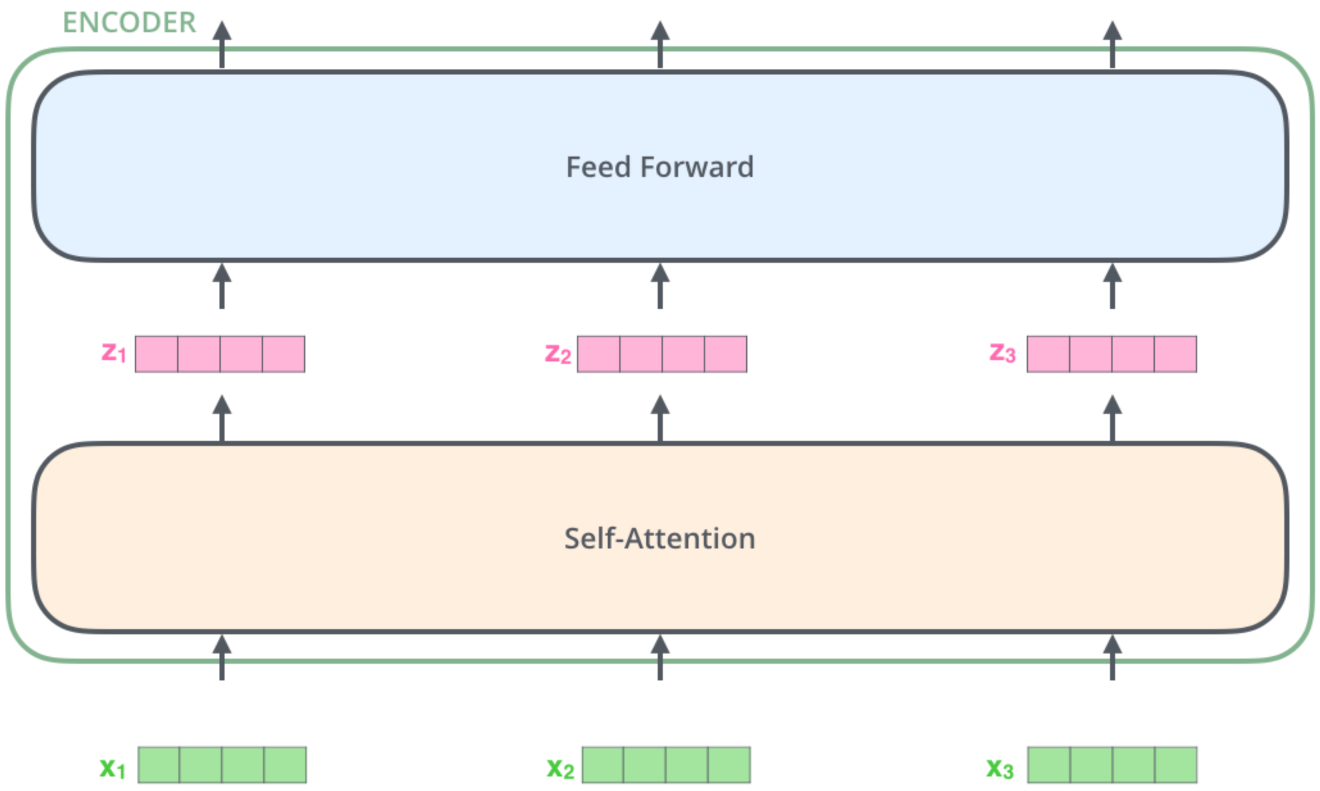
“keyanvexy” sequence length is 9!

No “keyanvexy” in

Zooming in on GPT models



High level picture of a transformer block



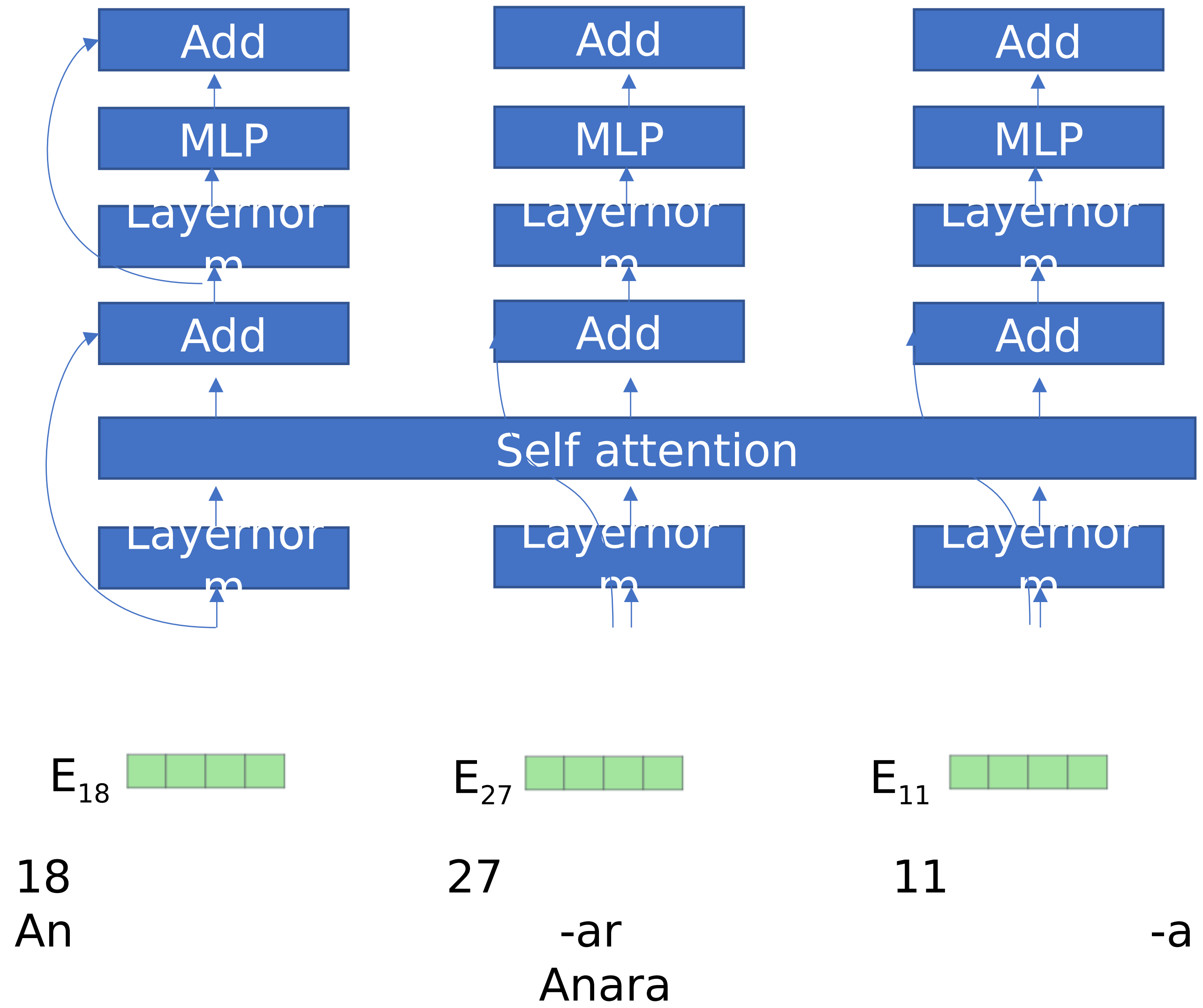
18
An
Anara

27
-ar

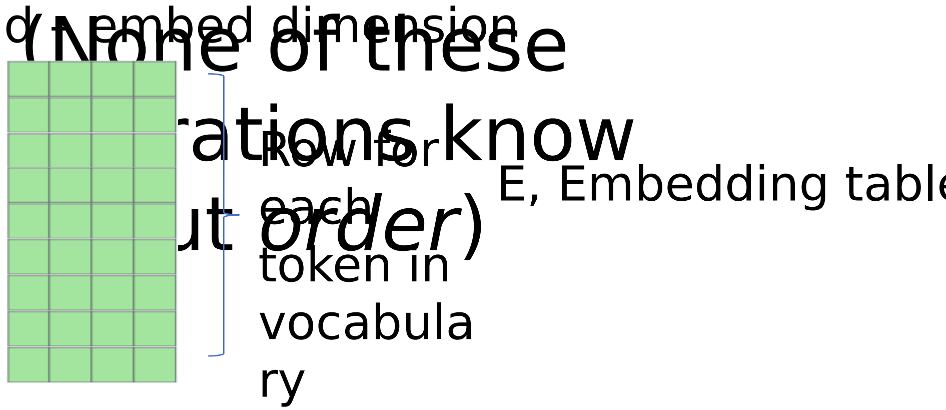
11
-a

- # parameters does not grow with sequence length
- Can process arbitrary length sequences, in principle
- Pad to get constant length during training

Better picture

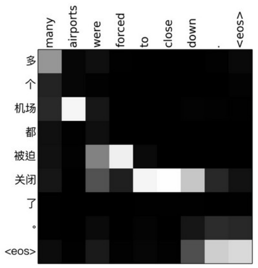


- Most parts are processed independently, in parallel EXCEPT Self-attention
- # parameters does not grow with sequence length
- Can process arbitrary length sequences, in principle
- (None of these ...rations know ...ut order)



Self-attention

First, split up embedding into three parts



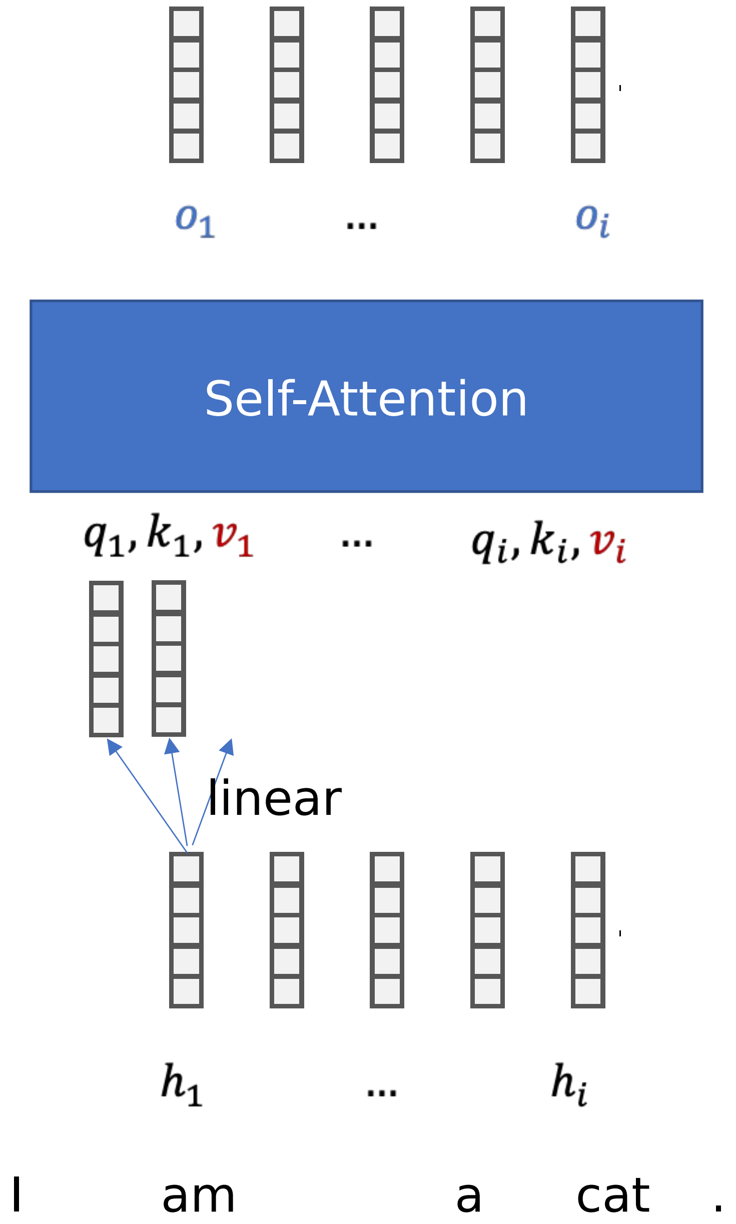
Output depends on a probability distribution over the inputs

$$o_i = \sum_j p(j|i) v_j$$

How do we get the probability distribution? We always use *softmax* to get discrete probabilities

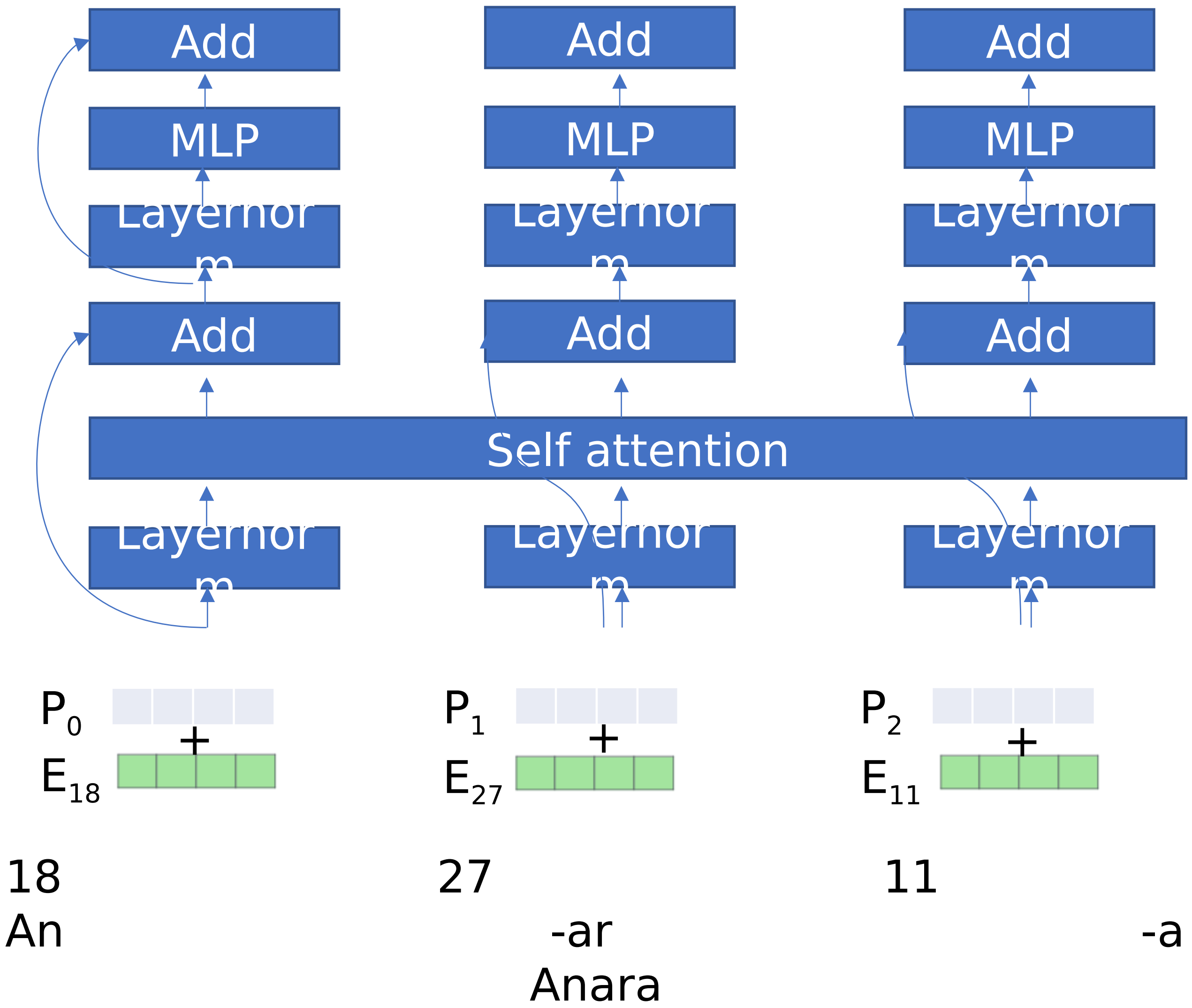
$$p(j|i) = softmax_j(q_i \cdot k_j) = \frac{e^{q_i \cdot k_j}}{\sum_{j'} e^{q_i \cdot k_{j'}}}$$

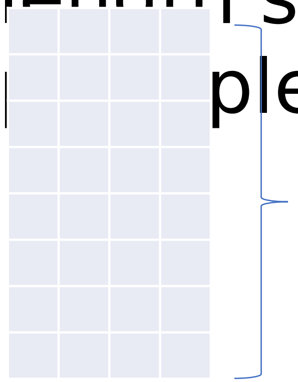
Position / order is not used
Permuting inputs would permute outputs



Position

- Most parts are processed independently, in parallel EXCEPT Self-attention
- # parameters does not grow with sequence length
- Can process arbitrary length sequences, in principle





Sinusoidal Positional Encoding

Sinusoidal positional encoding is defined as follows, given the token position $i = 1, \dots, L$ and the dimension $\delta = 1, \dots, d$:

$$\mathrm{PE}(i, \delta) = \begin{cases} \sin\left(\frac{i}{10000^{2\delta'/d}}\right) & \text{if } \delta = 2\delta' \\ \cos\left(\frac{i}{10000^{2\delta'/d}}\right) & \text{if } \delta = 2\delta' + 1 \end{cases}$$

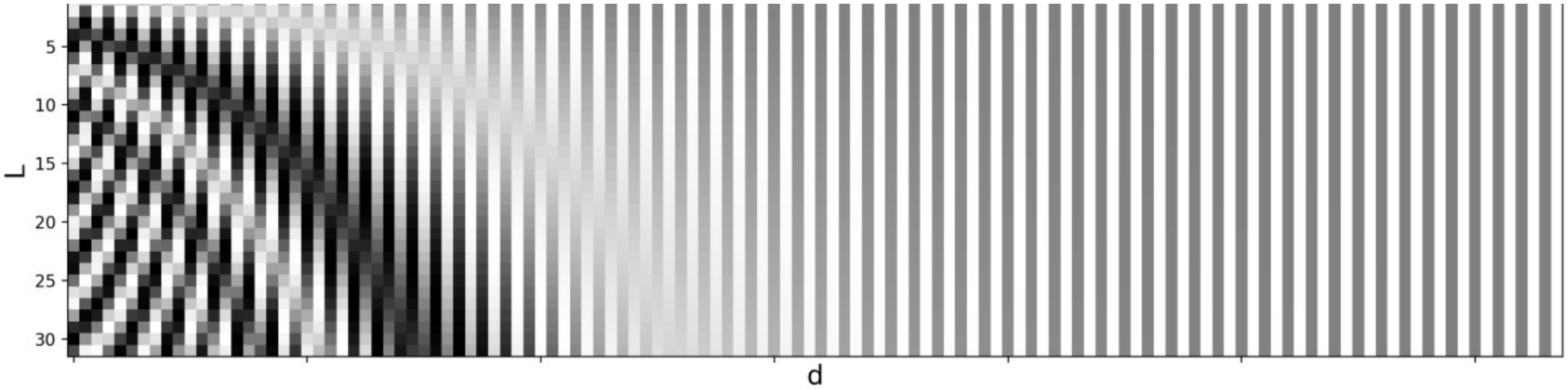
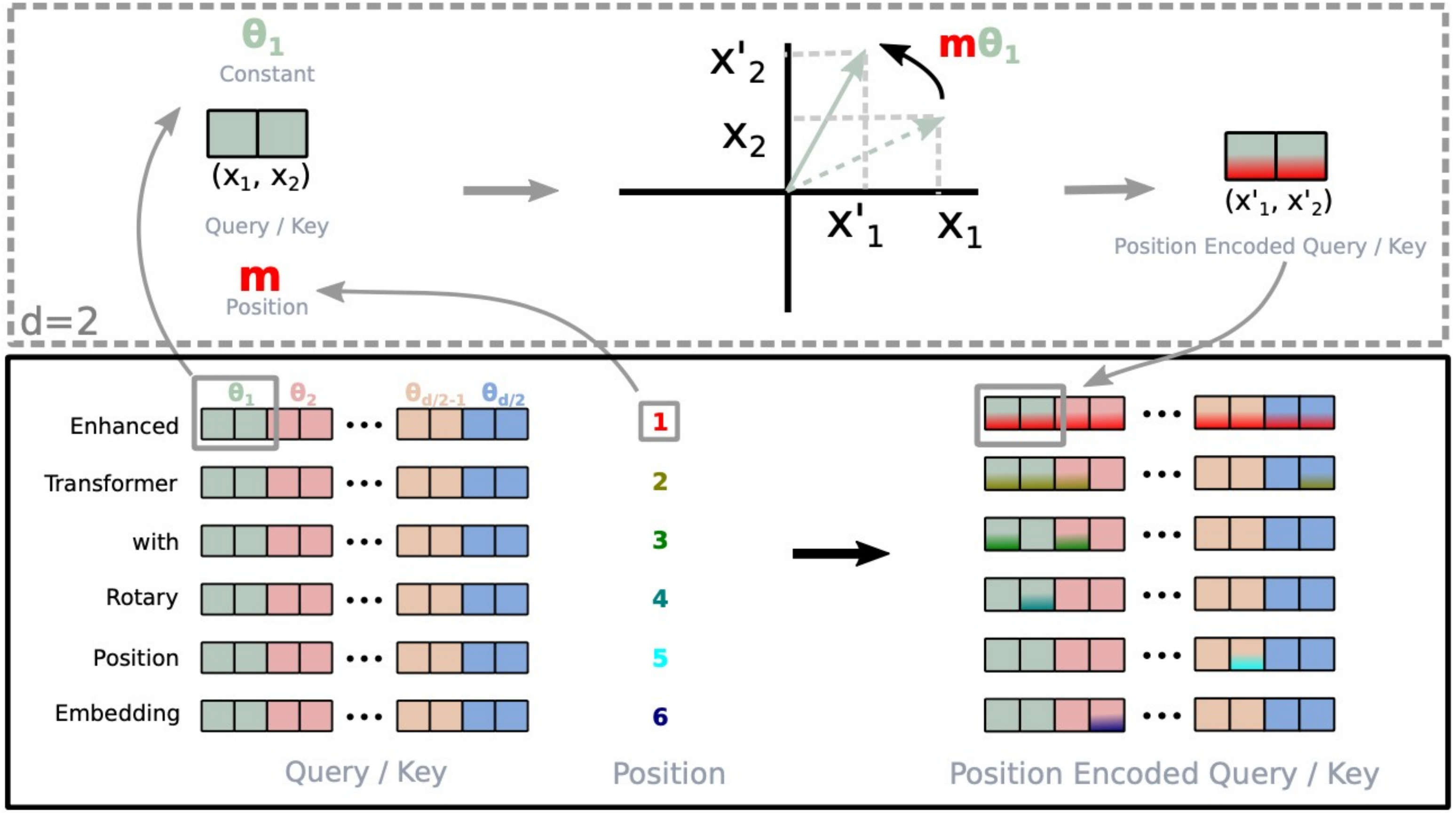


Fig. 3. Sinusoidal positional encoding with $L = 32$ and $d = 128$. The value is between -1 (black) and 1 (white) and the value 0 is in gray.

https://lilianweng.github.io/posts/2023-01-27-the-transformer-family-v2/

Newer work uses "rotation based" position embedding:

Rotation based position embedding?



https://arxiv.org/abs/2104.09864

Continue down the GPT rabbit hole next time